

ABSTRACT

With the increasing adoption of cloud computing platforms, organizations are migrating their identity and access management infrastructures to cloud environments such as Amazon Web Services (AWS). Despite this shift, credential-based attacks remain a significant security concern, particularly in environments that rely on Active Directory (AD) for authentication and authorization. Among these threats, password spray attacks pose a serious challenge due to their low-noise nature and ability to bypass traditional security controls.

This project presents the design and implementation of an **AWS-Based Active Directory Attack & Detection Lab** aimed at simulating a password spray attack and analyzing its detection using a Security Information and Event Management (SIEM) system. The lab environment is deployed within a secure AWS Virtual Private Cloud (VPC) and consists of multiple EC2 instances, including an Active Directory Domain Controller, a Windows 10 target workstation, a Kali Linux attacker machine, a Splunk Enterprise SIEM server, and a secure Jump Box for administrative access.

Password spray attacks are executed from the Kali Linux instance using industry-standard tools, while authentication events are generated and recorded by the Active Directory server. These events, including successful and failed login attempts, are forwarded to the Splunk SIEM platform for centralized log analysis and correlation. Detection is performed by analyzing Windows security event logs, specifically Event ID 4624 and Event ID 4625, enabling SOC-style investigation and validation of suspicious authentication behavior.

The project successfully demonstrates how cloud-hosted Active Directory environments can be monitored and secured using SIEM technologies. It provides practical exposure to cloud security architecture, attack simulation, and SOC operations, making it valuable for academic learning and foundational cybersecurity training.

INDEX

Sr. No.	Contents	Page No.
1	Introduction	
1.1	Introduction	
1.1.1	Problem Definition	
1.1.2	Objectives of Project	
1.1.3	Scope of Project	
1.2	Technical Details	
1.2.1	Overview of Front End	
1.2.2	Overview of Back End	
2	System Study and Planning	
2.1	System Study	
2.1.1 ¹	Existing System ²	
2.1.2 ³	Disadvantages of Existing System ⁴	
2.1.3 ⁵	Proposed System ⁶	
2.2 ⁷	System Planning and Schedule ⁸	
2.2.1 ⁹	S/W Development Model ¹⁰	
3¹¹	System Design	
3.1 ¹³	Software Requirement Specification (SRS) ¹⁴	
3.1.1 ¹⁵	Introduction of SR ¹⁶ S	
3.1.2	Technology Requirements	
3.1.2.1	Hardware to be used	
3.1.2.2	Software/tools to be used	
3.2	Detailed Life Cycle of the Project	
3.2.1	Modules	
3.2.2	Object Oriented Analysis & Design Diagrams	

3.2.2.1	Use Case Diagram	
3.2.2.2	Activity Diagram	
3.2.2.3	Class Diagram	
3.2.2.4	Sequence Diagram	
3.2.2.5	Flowchart / DFD / ER Diagram	
3.2.3.1	I/O Screen Layout	
4	Testing	
4.1	Methodologies used for Testing	
4.2	Types of Testing (Whichever Used)	
5	Conclusion	
6	Limitations	
7	Future Enhancements	
8	References	

1. Introduction

1.1 Introduction

In the modern digital era, organizations are rapidly adopting cloud computing platforms such as Amazon Web Services (AWS) to host their critical IT infrastructure. Despite this migration to the cloud, **identity and access management** remains a core security concern. Active Directory (AD) continues to be widely used as the primary authentication and authorization mechanism in enterprise environments, even when deployed on cloud infrastructure.

Credential-based attacks are among the most common and impactful cyber threats faced by organizations today. One such attack technique is the **password spray attack**, where an attacker attempts a small set of commonly used passwords across a large number of user accounts. This approach allows attackers to bypass account lockout policies and remain undetected for extended periods.

The project titled “**AWS-Based Active Directory Attack & Detection Lab**” aims to simulate a real-world password spray attack in a controlled AWS environment and analyze how such attacks can be detected using a Security Information and Event Management (SIEM) solution. This project combines concepts of cloud computing, cybersecurity, and SOC (Security Operations Center) operations, making it highly relevant for academic and industry-oriented learning.

1.1.1 Problem Definition

Organizations hosting Active Directory in cloud environments face several challenges in detecting credential-based attacks. Password spray attacks are particularly difficult to identify because they generate minimal noise and often appear as legitimate authentication failures.

The key problems addressed in this project include:

- Lack of centralized visibility into authentication events
- Difficulty in correlating multiple failed and successful login attempts
- Limited hands-on exposure to cloud-based SOC environments for students
- Delayed detection of low-and-slow credential attacks

This project addresses these issues by designing a cloud-hosted lab that enables realistic attack simulation and effective detection using SIEM technology.

1.1.2 Objectives of the Project

The primary objectives of this project are:

- To design and deploy an Active Directory-based lab environment on AWS
- To simulate password spray attacks using a Kali Linux attacker machine
- To generate security telemetry using Windows event logs, Sysmon, and Atomic Red Team
- To collect and analyze logs using Splunk Enterprise SIEM
- To study authentication-related Windows Event IDs such as 4624 and 4625

- To understand SOC-level detection and investigation workflows in cloud environments

1.1.3 Scope of the Project

The scope of this project includes:

- Deployment of virtual machines using AWS EC2
- Configuration of Active Directory Domain Services on Windows Server
- Use of Windows 10 as a target workstation for telemetry generation
- Simulation of attacks using Kali Linux
- Centralized log collection and analysis using Splunk SIEM

The project does not include advanced exploitation techniques such as privilege escalation, lateral movement, or integration with managed AWS security services. The focus is strictly on attack simulation and detection.

1.2 Technical Details

1.2.1 Overview of Front End

The front end of the system is represented by the **Splunk Enterprise Web Interface**, which provides SOC analysts with the ability to search, visualize, and analyze security events. The front end includes:

- Interactive dashboards for authentication monitoring
- Search Processing Language (SPL) queries for log analysis
- Visualization of failed and successful login attempts
- Alerting mechanisms for suspicious behavior

The Splunk web interface serves as the primary interaction point for monitoring and investigation.

1.2.2 Overview of Back End

Role of Individual EC2 Instances in the Project

The AWS-based lab environment is composed of multiple EC2 instances, each assigned a specific role to accurately simulate a real-world enterprise and SOC-monitored infrastructure. The responsibilities of each instance are described below:

1. Active Directory (AD) Server – Windows Server EC2

The Active Directory instance acts as the core identity and authentication service of the environment. It functions as the Domain Controller and is responsible for managing domain users, validating login attempts, enforcing authentication policies, and generating Windows Security Event Logs. During the password spray attack simulation, this server processes all authentication requests and generates critical events such as Event ID 4625 (failed logon) and Event ID 4624 (successful logon), which form the primary detection data for the project.

2. Kali Linux Attacker Instance – EC2

The Kali Linux instance represents an external threat actor. It is intentionally placed within the same virtual network to simulate an internal or VPN-connected attacker scenario. Tools

such as NetExec are executed from this machine to perform SMB-based password spray attacks against Active Directory user accounts. The purpose of this instance is to generate realistic malicious authentication traffic without affecting production systems.

3. Splunk Enterprise Server – EC2

The Splunk server acts as the centralized Security Information and Event Management (SIEM) platform. It receives logs forwarded from the Active Directory server and Windows endpoints, indexes them, and enables correlation and analysis using SPL queries. SOC analysts use this instance to monitor authentication activity, detect abnormal login patterns, and investigate potential security incidents resulting from the password spray attack.

4. Jump Box Server – Windows EC2

The Jump Box server serves as a secure administrative access point to the AWS environment. It is used by administrators and SOC analysts to remotely access internal instances such as the Active Directory server and Splunk server without exposing them directly to the internet. This setup reflects real-world security best practices by reducing the attack surface and centralizing privileged access management.

AWS Infrastructure Configuration Details

Virtual Private Cloud (VPC) Configuration:

A dedicated Amazon Virtual Private Cloud (VPC) was created to host all EC2 instances involved in the project. The VPC provides logical isolation of the lab environment from other AWS resources. A private IPv4 CIDR block was assigned to the VPC to allow internal communication between instances. This design ensures that authentication traffic, attack simulations, and log forwarding occur within a controlled and isolated network.

Subnets:

The VPC was divided into public and private subnets. The Jump Box and Kali Linux attacker instances were placed in a public subnet with controlled internet access, while the Active Directory server, Windows 10 target machine, and Splunk server were deployed in private subnets. This segmentation improves security by restricting direct external access to critical servers.

Internet Gateway and Routing:

An Internet Gateway was attached to the VPC to enable controlled outbound and inbound internet access for instances located in the public subnet. Route tables were configured to ensure that only authorized instances could communicate with external networks, while internal traffic between private instances remained isolated within the VPC.

Security Groups:

Security Groups were configured as stateful virtual firewalls to regulate traffic between EC2 instances. Specific inbound rules were defined to allow only required services such as SMB, RDP, HTTP, and Splunk management ports. This minimized the attack surface and ensured that unnecessary network exposure was avoided.

Identity and Access Management (IAM):

AWS Identity and Access Management (IAM) was used to enforce least-privilege access control. IAM roles were assigned to EC2 instances to allow secure interaction with AWS services without hardcoded credentials. IAM users were created with restricted permissions for administrative tasks, ensuring accountability and secure access management.

Logging and Monitoring:

AWS-level monitoring was enabled using CloudWatch to observe instance health and performance metrics. While the primary focus of the project is on Windows authentication logs, CloudWatch provides additional visibility into infrastructure-level events, enhancing overall situational awareness.

The back end consists of multiple AWS-hosted components working together to generate and process security telemetry:

- AWS EC2 instances hosting Windows Server (Active Directory)
- Windows 10 EC2 instance acting as a target machine
- Kali Linux EC2 instance used as an attacker machine
- Splunk Enterprise EC2 instance for centralized log analysis
- Windows Event Logs and Sysmon logs forwarded to Splunk

This back-end architecture closely resembles a real-world cloud SOC environment.

Instances (5) [Info](#)

All states ▾

☐

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐ DC01	i-0f74dd942fc572ff8	Running	t3.small	3/3 checks passed	View alarms	ap-south-1a
☐ Win-client	i-06fc3ff8ac3a94a3a	Running	t3.small	3/3 checks passed	View alarms	ap-south-1a
☐ Splunk-Server	i-0fe37d4e3e55d875e	Running	t3.small	3/3 checks passed	View alarms	ap-south-1a
☐ Kali-Attacker	i-0c42178fb7dbdd373	Running	t3.small	3/3 checks passed	View alarms	ap-south-1a
☐ JumpBox	i-06a68018ab41918c7	Running	t3.micro	3/3 checks passed	View alarms	ap-south-1a

Security Groups (4) [Info](#)

☐

Name	Security group ID	Security group name
☐ -	sg-097aa1ce0d15240ad	jumpbox-sg
☐ -	sg-008b9076193c72640	default
☐ -	sg-06429e707cebd73d2	default
☐ new-lab-SG	sg-0021f9801d32b349a	DFIR-Lab-SG

Your VPCs

VPCsVPC encryption controls

Your VPCs (1/2)Info

Find VPCs by attribute or tag

	Name	VPC ID	State
☐	-	vpc-04e2e1e2b4868dba7	Available
☒	lab-vpc	vpc-097b33819f2a9dc9e	Available

vpc-097b33819f2a9dc9e / lab-vpc

DetailsResource mapCIDRsFlow logsTagsIntegrations

Details

VPC ID

vpc-097b33819f2a9dc9e

DNS resolution

Enabled

Main network ACL

[acl-01040eec475c3046d](#)

IPv6 CIDR (Network border group)

-

Encryption control ID

-

State

Available

Tenancy

default

Default VPC

No

Network Address Usage metrics

Disabled

Encryption control mode

-

lab-SSM-RMInfo

Allows EC2 instances to call AWS services on your behalf.

Summary

Creation date

December 07, 2025, 17:44 (UTC+05:30)

ARN

arn:aws:iam::116360783295:role/lab-SSM-RM

Last activity

9 minutes ago

Maximum session duration

1 hour

PermissionsTrust relationshipsTagsLast AccessedRevoke sessions

Permissions policies (1)Info

You can attach up to 10 managed policies.

Search

Filter by Type

All types

Policy name	Type
AmazonSSMManagedInstanceCore	AWS managed

2. System Study and Planning

2.1 System Study

System study involves a detailed analysis of the existing authentication monitoring mechanisms and the identification of gaps that may lead to delayed detection of credential-based attacks. This phase helps in understanding the operational shortcomings of traditional environments and forms the foundation for proposing a more effective cloud-based solution.

2.1.1 Existing System

In many traditional on-premise enterprise environments, authentication and security logs are stored locally on individual systems such as domain controllers, servers, and workstations. Security teams rely heavily on manual inspection of Windows Event Logs or basic alerting mechanisms provided by native operating system tools.

These environments typically lack centralized log aggregation and correlation capabilities. As a result, detecting stealthy attacks such as password spray attacks becomes difficult, since failed authentication attempts are distributed across multiple systems and time intervals. Furthermore, traditional systems are not designed to scale efficiently or integrate seamlessly with cloud-native security solutions.

2.1.2 Disadvantages of Existing System

The existing system suffers from several critical limitations:

- **No Centralized Log Aggregation:**
Logs are stored locally on individual machines, making it difficult to correlate events across multiple endpoints.
- **High Dependency on Manual Analysis:**
Security analysts must manually review logs, which increases operational effort and the probability of human error.
- **Delayed Detection of Credential-Based Attacks:**
Attacks such as password spraying often remain undetected for extended periods due to low alerting sensitivity.

- **Limited Scalability and Flexibility:**
Traditional systems struggle to handle increased users, devices, and authentication events.
- **Inadequate Training Environments for SOC Analysts:**
On-premise setups do not provide realistic, cloud-aligned training exposure required for modern SOC roles.

2.1.3 Proposed System

The proposed system introduces a **cloud-based security lab hosted on Amazon Web Services (AWS)** that enables centralized monitoring and realistic attack detection. The key features of the proposed system include:

- Deployment of a **cloud-hosted Active Directory Domain Controller** using AWS EC2.
- Centralized log collection and correlation using **Splunk Enterprise SIEM**.
- Simulation of real-world attacks using a **Kali Linux attacker instance**.
- Secure administrative access through a **Jump Box server**.
- Enhanced visibility into authentication events such as failed and successful logins.

This system significantly improves detection accuracy, scalability, and provides hands-on experience with cloud-based SOC operations.

2.2 System Planning and Schedule

The project was planned and executed in a structured and phased manner to ensure proper implementation and validation. The following stages were followed:

1. Requirement analysis and project planning
2. AWS infrastructure design and VPC configuration
3. EC2 instance deployment and network setup
4. Active Directory installation and configuration
5. Log forwarding and telemetry setup using Sysmon and Splunk Forwarder

6. Attack simulation using Kali Linux
7. Detection, investigation, and analysis using Splunk
8. Documentation and final reporting

This systematic planning ensured smooth execution and minimized configuration errors.

2.2.1 Software Development Model

For this project, the **Agile Software Development Model** was adopted due to its flexibility, iterative approach, and suitability for projects where requirements may evolve during development. Unlike traditional sequential models, Agile emphasizes incremental delivery, continuous feedback, and adaptive planning, which aligns well with the dynamic nature of cloud-based cybersecurity labs and SOC-style experimentation.

The project was structured into **multiple iterative sprints**, each focusing on a specific module or functional component of the lab. The key sprints included:

1. **Requirement Analysis and Planning Sprint:**
In this sprint, the team analyzed functional and non-functional requirements, including AWS infrastructure needs, Active Directory setup, and SIEM integration. This phase helped prioritize critical tasks for subsequent iterations.
2. **AWS Infrastructure Deployment Sprint:**
The AWS Virtual Private Cloud (VPC), subnets, EC2 instances, security groups, and IAM roles were configured. The iterative approach allowed verification of connectivity, security, and accessibility at each stage.
3. **Active Directory Configuration Sprint:**
Domain services, users, groups, and policies were configured. Iterative testing ensured correct authentication flows and proper log generation for monitoring.
4. **Target Workstation and Attacker Module Sprint:**
Windows 10 target machines were configured with Sysmon and Splunk Forwarder, while the Kali Linux attacker instance was set up with necessary tools. Each iteration included testing attack simulations and verifying telemetry generation.
5. **Log Collection, SIEM Integration, and Detection Sprint:**
Splunk Enterprise dashboards, log indexing, and SPL queries were implemented and refined iteratively. Each iteration allowed the team to validate detection accuracy and optimize event correlation logic.
6. **Testing and Feedback Sprint:**
Functional and security testing was conducted iteratively, with findings feeding back into previous sprints for improvements. This ensured a high-quality and fully validated environment.

7. **Documentation Sprint:**

Throughout the project, documentation was maintained incrementally, reflecting changes and updates made during each sprint. This ensured consistency between implementation and report content.

Advantages of Agile in This Project

- **Iterative Delivery:** Enabled testing and validation of each component progressively rather than waiting until the end, reducing errors and rework.
- **Flexibility:** Allowed the team to incorporate changes in AWS configurations, attack simulations, or SIEM settings without disrupting the entire project.
- **Continuous Feedback:** Iterative testing and instructor feedback helped improve detection strategies and lab configurations in real-time.
- **Enhanced Collaboration:** Agile encouraged collaboration between project members, fostering discussion on optimal lab design, attack simulations, and monitoring techniques.
- **Transparency and Tracking:** Progress was visible after each sprint, making evaluation and grading more straightforward.

Overall, the Agile methodology enhanced **efficiency, adaptability, and learning outcomes**, making it highly suitable for an academic project focused on cloud-based cybersecurity operations and practical SOC training.

.

3. System Design

3.1 Software Requirement Specification (SRS)

3.1.1 Introduction of SRS

The Software Requirement Specification (SRS) document defines both the functional and non-functional requirements of the **AWS-based Active Directory Attack and Detection Lab**. It serves as a blueprint for system development and ensures that the final implementation aligns with project objectives.

3.1.2 Technology Requirements

3.1.2.1 Hardware to be Used

- Host system with a minimum of **16 GB RAM**
- Minimum **250 GB disk space**
- Stable broadband internet connection

3.1.2.2 Software / Tools to be Used

- **Amazon Web Services (AWS):** EC2, VPC, Security Groups, IAM
- **Windows Server:** Active Directory Domain Services
- **Windows 10:** Target workstation
- **Kali Linux:** Attacker machine
- **Splunk Enterprise:** SIEM platform
- **Sysmon:** Endpoint telemetry generation
- **Atomic Red Team:** Attack simulation framework
- **NetExec (nxc):** Credential-based attack execution

3.2 Detailed Life Cycle of the Project

The project lifecycle includes the following phases:

- Requirement gathering and feasibility analysis
- AWS infrastructure and network design
- EC2 instance deployment
- Active Directory and domain configuration
- Log forwarding and SIEM integration
- Attack execution and monitoring
- Detection validation and result analysis
- Documentation and reporting

Each stage was executed sequentially to ensure completeness and reliability.

3.2.1 Modules

The system is divided into the following functional modules:

1. AWS Infrastructure Module

Responsible for configuring VPC, subnets, routing, security groups, IAM roles, and EC2 instances to ensure secure and isolated communication.

2. Active Directory Module

Implements identity management, user authentication, and policy enforcement. The Domain Controller generates authentication logs essential for attack detection.

3. Target Workstation Module

Acts as a legitimate domain-joined endpoint generating user authentication activity and forwarding logs to Splunk.

4. Attacker Module

Simulates real-world adversary behavior using Kali Linux and tools like Nmap, Kerbrute, and NetExec.

5. Log Collection and SIEM Module

Centralizes logs using Splunk Enterprise and enables detection, correlation, and investigation of security events.

Commands Used and Their Purpose

RDP Access via AWS SSM

Used for secure administrative access without exposing RDP ports publicly.

```
aws ssm start-session --target <instance-id> \  
--document-name AWS-StartPortForwardingSession \  
--parameters '{"portNumber":["3389"], "localPortNumber":["3390"]}'
```

Splunk Universal Forwarder Installation

Used to forward Windows logs from endpoints to Splunk SIEM.

```
Invoke-WebRequest -Uri "https://download.splunk.com/..." -OutFile  
"SplunkUniversalForwarder.msi"
```

Attack Simulation Commands

- Network Scanning

```
sudo nmap -p- -sV 10.0.1.100
```

- Password Spray Attack

```
kerbrute passwordspray -d lab.local --dc 10.0.1.100 users.txt 'Password123!'
```

Detection Query in Splunk

```
index=main sourcetype="WinEventLog:Security" EventCode=4625  
| stats count by _time, TargetUserName, Source_Network_Address, Logon_Type
```

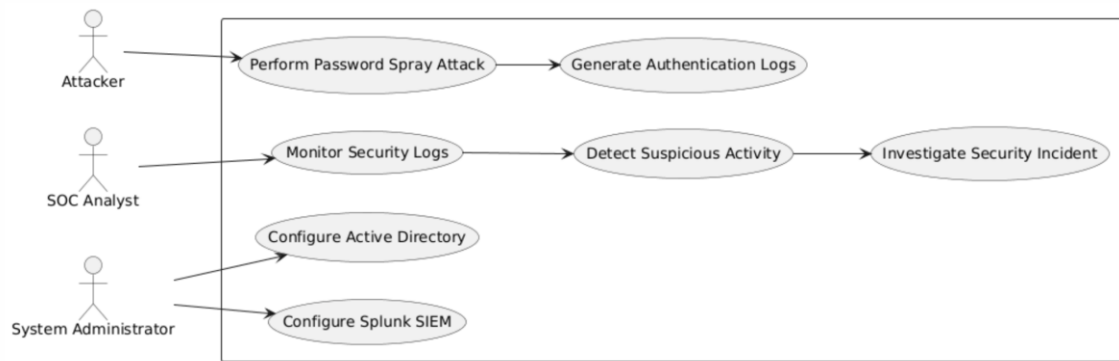
This query identifies multiple failed authentication attempts, a key indicator of password spray attacks.

3.1.2 Object Oriented Analysis & Design Diagrams

3.1.2.1 Use Case Diagram

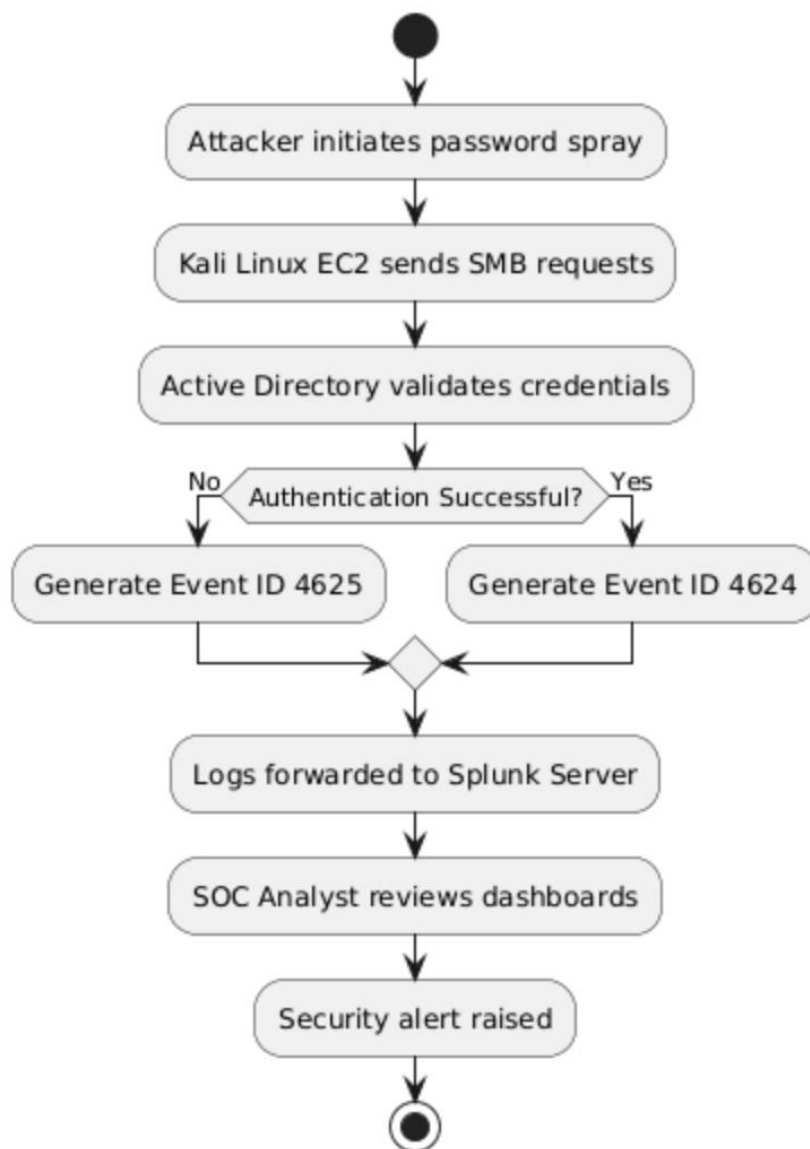
This diagram illustrates the interaction between the attacker and the SOC analyst within the system.

Symbol	Name	Description
	Use Case	Use cases are used to represent high-level functionalities and how the user will handle the system. A use case represents a distinct functionality of a system, a component, a package, or a class. It is denoted by an oval shape with the name of a use case written inside the oval shape.
	Actor	It is used inside use case diagrams. The actor is an entity that interacts with the system. A user is the best example of an actor. An actor is an entity that initiates the use case from outside the scope of a use case. It can be any element that can trigger an interaction with the use case. One actor can be associated with multiple use cases in the system.



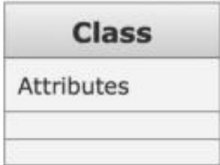
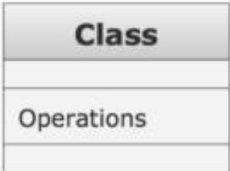
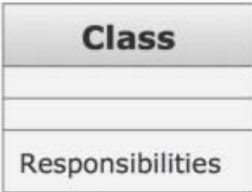
3.2.2.2 Activity Diagram

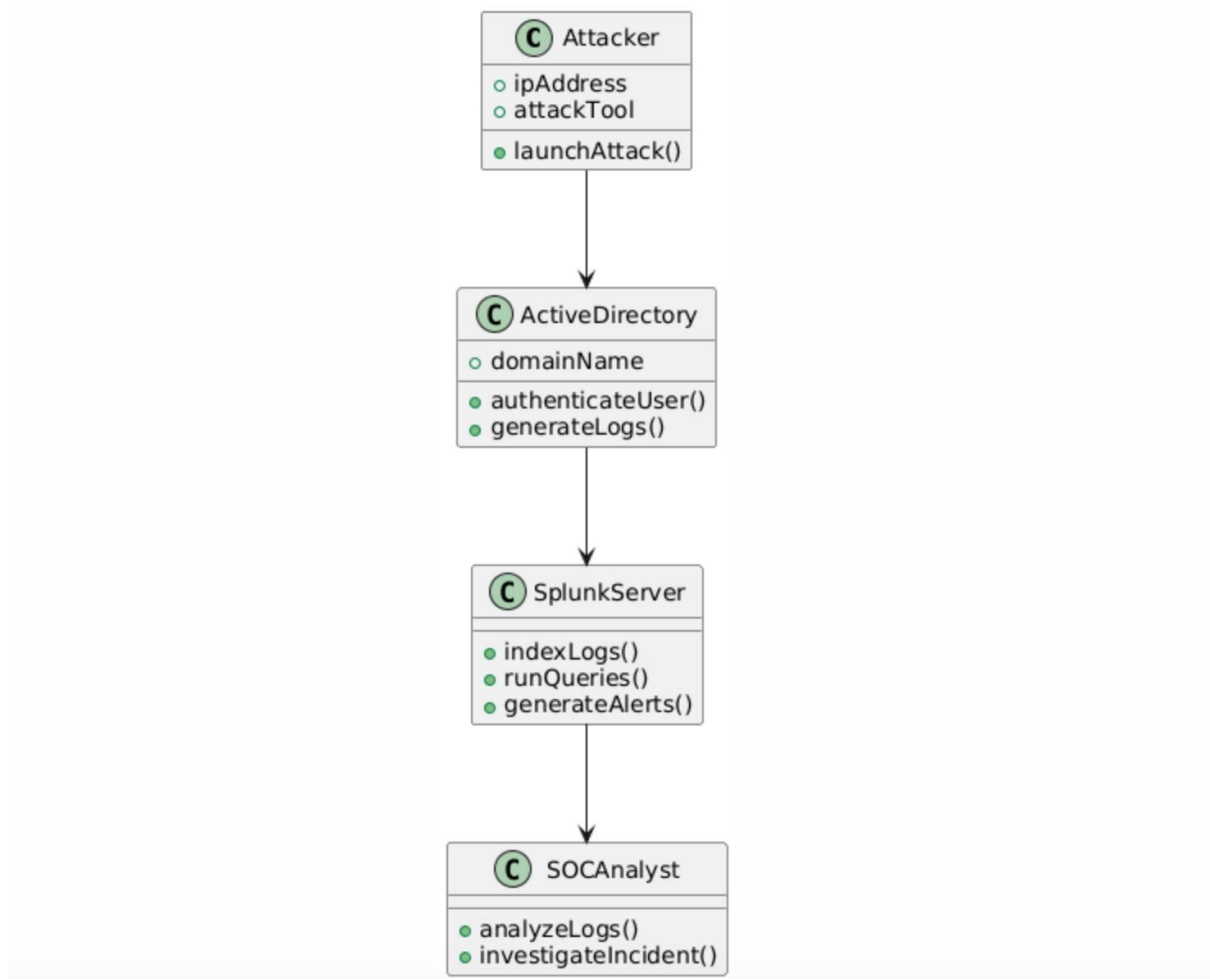
The activity diagram represents the flow of actions from attack execution to detection.



3.2.2.3 Class Diagram

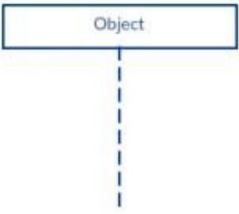
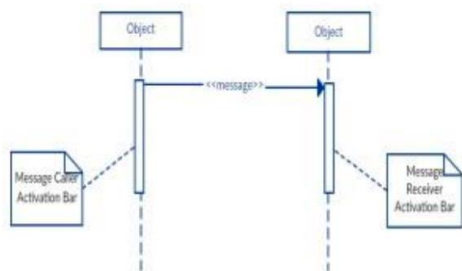
The class diagram shows the relationship between users, authentication events, and SIEM components.

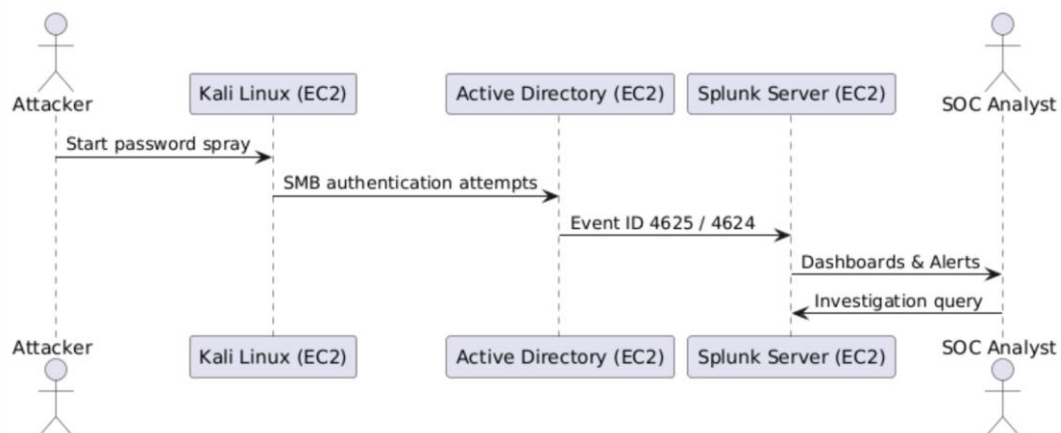
Symbol	Name	Description
	Class	Class represents a set of objects that have the same structure, behavior, and relationships with objects of other classes.
	Attribute	Attribute is a typed value that defines the properties and behavior of the object.
	Operation	Operation is a function that can be applied to the objects of a given class.
	Responsibility	Responsibility is a contract which the class must conform.
	Association	Association is a relationship that connect two classes.



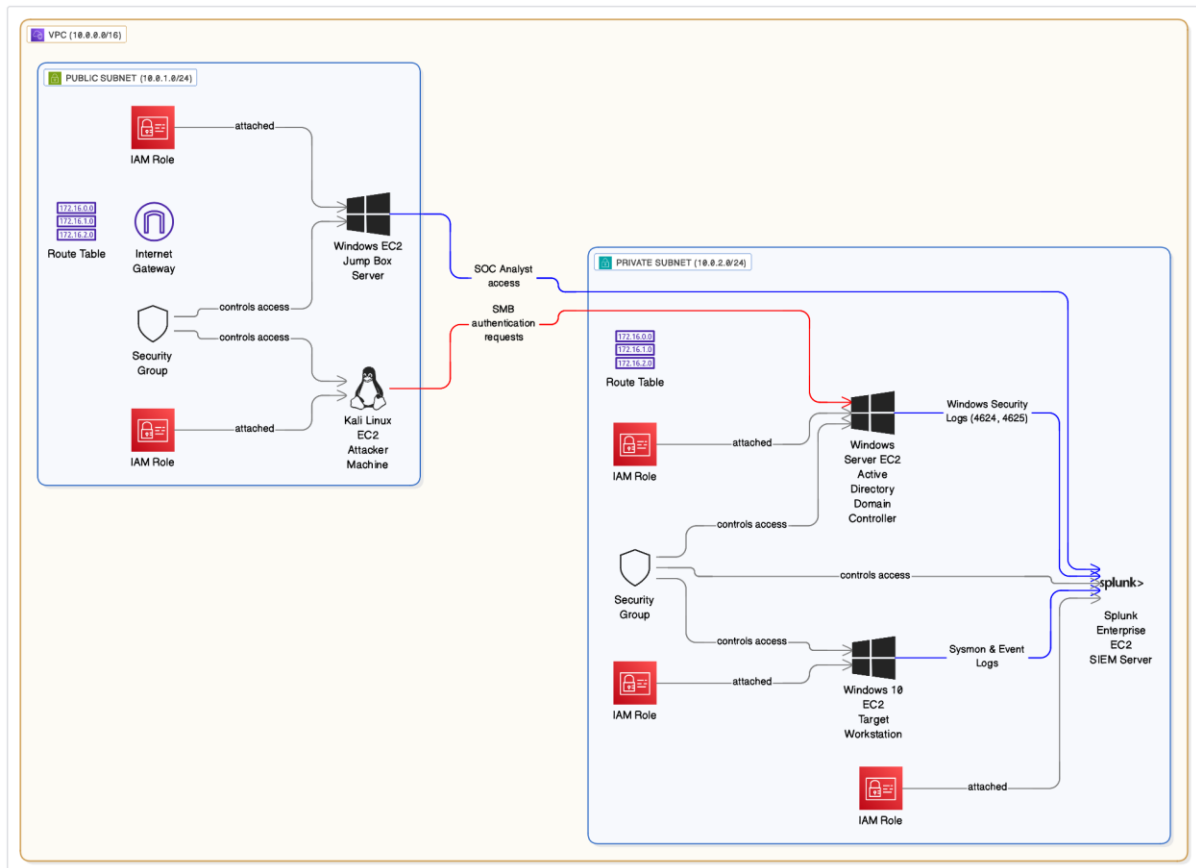
3.2.2.4 Sequence Diagram

The sequence diagram illustrates the order of interactions between system components.

Symbol	Name	Description
	Lifeline Notation	A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram. No two lifeline notations should overlap each other.
	Activation Bars	Activation bar is the box placed on the lifeline. It is used to indicate that an object is active (or instantiated) during an interaction between two objects. The length of the rectangle indicates the duration of the objects staying active. In a sequence diagram, an interaction between two objects occurs when one object sends a message to another.
	Synchronous Message	A synchronous message is used when the sender waits for the receiver to process the message and return before carrying on with another message. The arrowhead used to indicate this type of message is a solid one.



3.2.2.5 AWS Architecture diagram



3.2.3.1 I/O Screen Layout

Input:

- Username list
- Password list

Output:

- Authentication success and failure logs
- Splunk dashboards and alerts

4. Testing

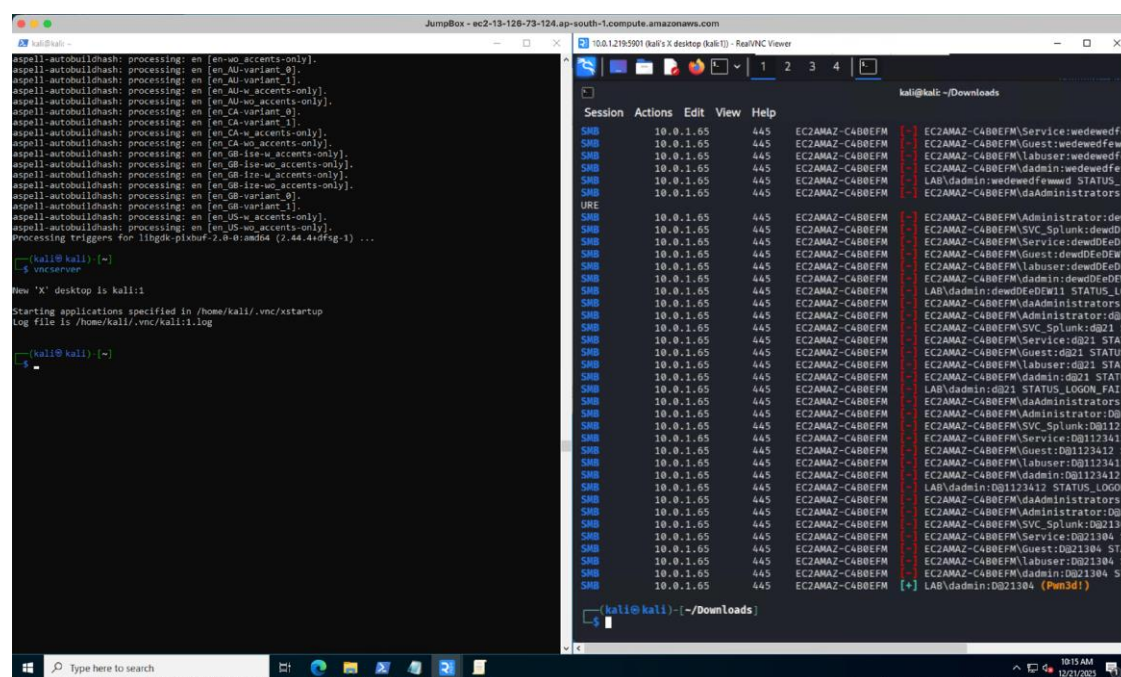
4.1 Methodologies Used for Testing

Black-box testing methodology was used in this project. The internal system logic was not considered while testing; instead, focus was placed on validating expected outputs based on given inputs.

4.2 Types of Testing

- Functional Testing
- Security Testing
- Log Validation Testing

Jump box + kali output :

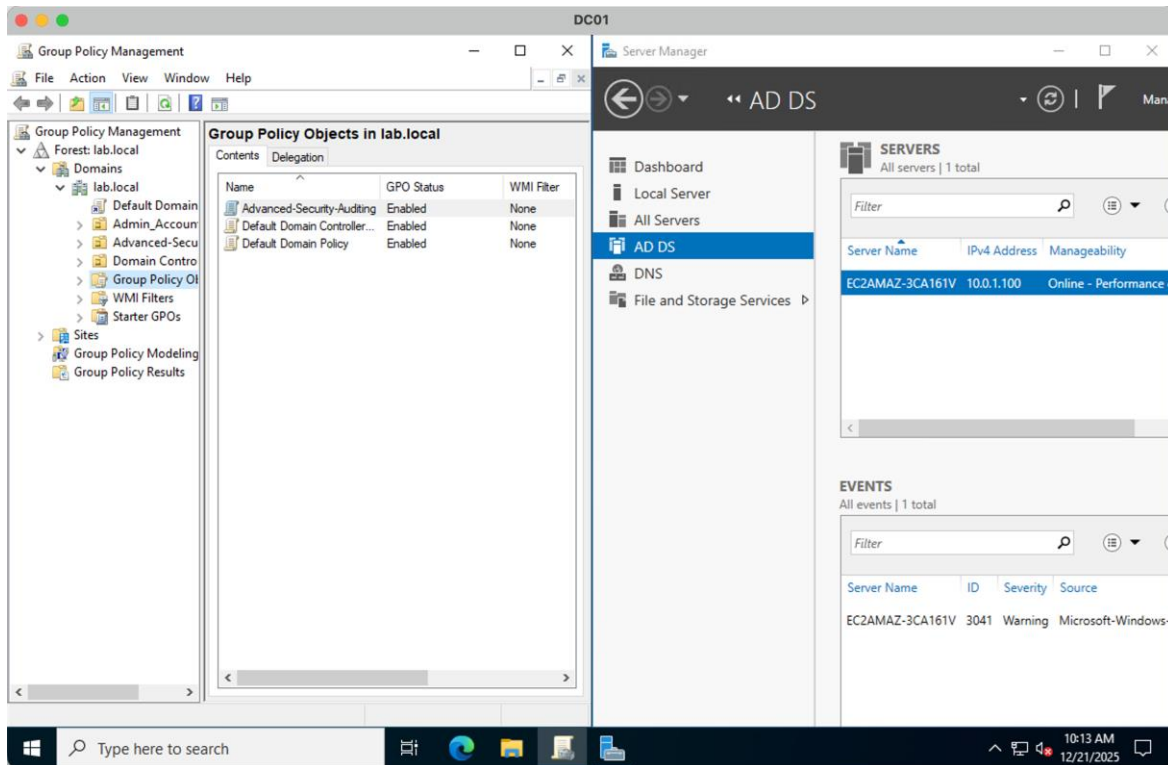


```
aspell-autobuildhash: processing: en [en-wo_accents-only].
aspell-autobuildhash: processing: en [en_AU-variant-0].
aspell-autobuildhash: processing: en [en_AU-variant-1].
aspell-autobuildhash: processing: en [en_AU-w_accents-only].
aspell-autobuildhash: processing: en [en_AU-wo_accents-only].
aspell-autobuildhash: processing: en [en_CA-variant-0].
aspell-autobuildhash: processing: en [en_CA-variant-1].
aspell-autobuildhash: processing: en [en_CA-wo_accents-only].
aspell-autobuildhash: processing: en [en_GB-lse-w_accents-only].
aspell-autobuildhash: processing: en [en_GB-lse-wo_accents-only].
aspell-autobuildhash: processing: en [en_GB-lse-w_accents-only].
aspell-autobuildhash: processing: en [en_GB-lse-wo_accents-only].
aspell-autobuildhash: processing: en [en_GB-variant-0].
aspell-autobuildhash: processing: en [en_GB-variant-1].
aspell-autobuildhash: processing: en [en_US-w_accents-only].
aspell-autobuildhash: processing: en [en_US-wo_accents-only].
Processing triggers for lingo-pixbuf-2.0-0:amd64 (2.44.4+dfsg-1) ...

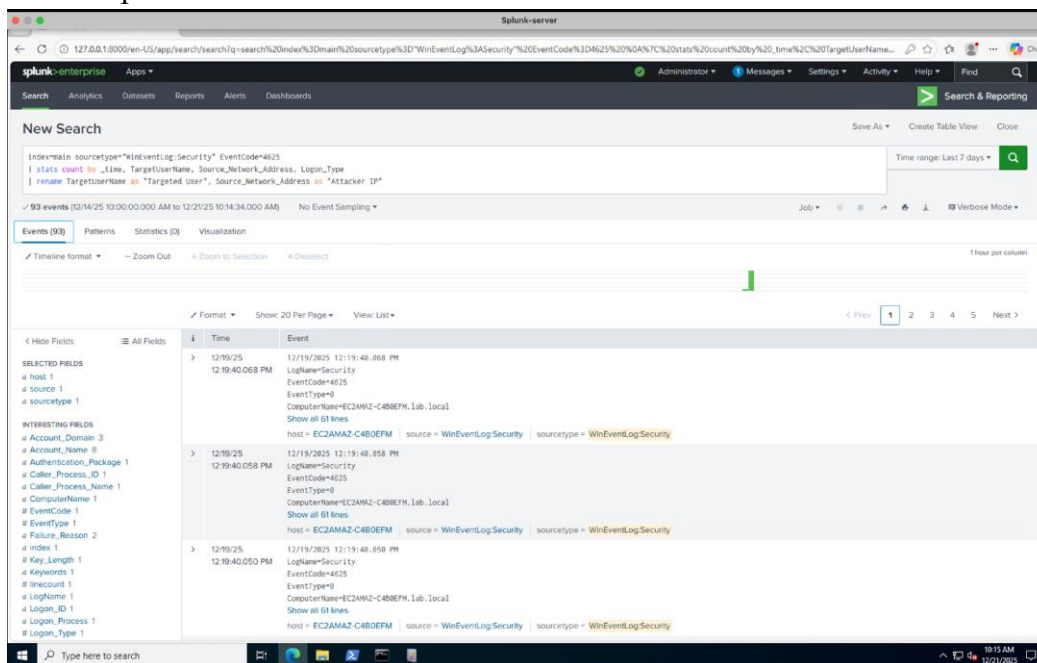
[kali@kali] ~$ vncserver
New 'X' desktop is kali:1
Starting applications specified in /home/kali/.vnc/xstartup
log file is /home/kali/.vnc/kali:1.log

[kali@kali] ~$
```

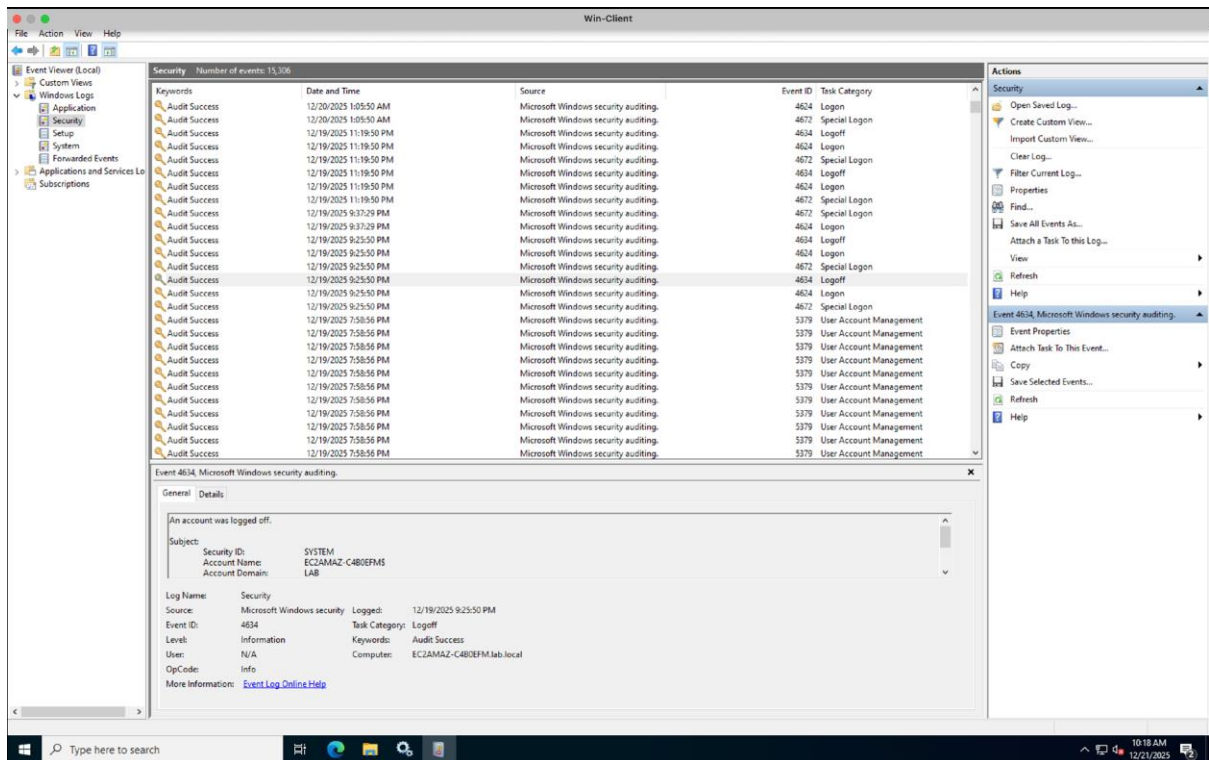
DC01



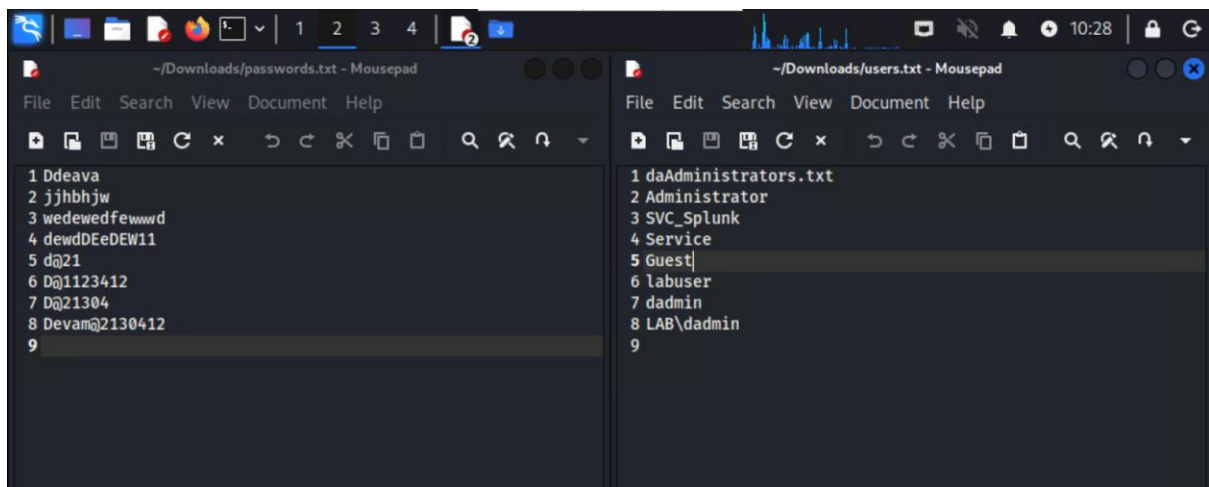
Splunk server



• Win client



Kali :



5. Conclusion

- This project successfully demonstrates the design, implementation, and evaluation of an **AWS-based Active Directory Attack and Detection Lab**, focusing on the simulation and detection of password spray attacks. By deploying a cloud-hosted Active Directory environment within a secure AWS Virtual Private Cloud (VPC), the project effectively replicates a real-world enterprise authentication infrastructure.
- The execution of password spray attacks using a dedicated Kali Linux attacker instance enabled the generation of realistic authentication telemetry. The Active Directory Domain Controller played a critical role in validating authentication attempts and generating security event logs, including both successful and failed logon events. These logs were centrally collected and analyzed using **Splunk Enterprise SIEM**, providing enhanced visibility into authentication activities across the environment.
- Through the correlation of multiple failed login attempts followed by a successful authentication, the project demonstrated how low-noise credential-based attacks can be identified using SIEM technologies. The use of Splunk dashboards and Search Processing Language (SPL) queries allowed SOC-style investigation and validation of suspicious behavior.
- Overall, this project highlights the significance of **cloud-based SOC monitoring**, centralized log management, and proactive detection mechanisms in modern cybersecurity operations. It provides valuable practical exposure to cloud security architecture, identity-based attack detection, and security monitoring workflows, making it an effective learning platform for academic and foundational SOC training.

6. Limitations

- Despite successfully achieving its objectives, the project has certain limitations that should be acknowledged:
- **Limited Scale of the Lab Environment**

The lab environment is designed for academic and learning purposes and operates on a small scale with a limited number of user accounts and endpoints. As a result, it may not fully represent the complexity and traffic volume of large enterprise environments.
- **Absence of Multi-Factor Authentication (MFA)**

The project does not implement multi-factor authentication, which is a widely adopted security control in modern enterprises. The absence of MFA limits the ability to evaluate the effectiveness of additional authentication layers against credential-based attacks.
- **Single Attack Technique Simulation**

The project focuses solely on password spray attacks. Other attack techniques such as phishing, credential dumping, lateral movement, and privilege escalation are not included, limiting the scope of threat coverage.
- **Manual Investigation Process**

Detection and investigation activities rely on manual analysis using Splunk queries and dashboards. Automated incident response and remediation mechanisms are not implemented.

7. Future Enhancements

- The project can be extended and improved in several ways to increase its realism and effectiveness:
- **Integration with AWS GuardDuty and CloudTrail**
Future enhancements can include integration with AWS-native security services such as GuardDuty and CloudTrail. This would enable correlation between identity-based attacks and AWS infrastructure-level events, providing deeper visibility into cloud threats.
- **Implementation of Automated Alerts and SOAR**
Automated alerting and Security Orchestration, Automation, and Response (SOAR) capabilities can be implemented to reduce manual effort. Automated workflows could be used to trigger alerts, isolate compromised accounts, or notify administrators in real time.
- **Simulation of Additional Attack Techniques**
The lab can be expanded to include other attack techniques such as brute-force attacks, phishing simulations, lateral movement, and privilege escalation. This would allow comprehensive threat detection and SOC training.
- **Hybrid Active Directory Testing**
A hybrid Active Directory setup combining on-premise AD with Azure AD or AWS-hosted AD services can be implemented. This would help study identity security challenges in hybrid and multi-cloud environments.
- **Scalability and Performance Testing**
Increasing the number of users, endpoints, and authentication events would help evaluate SIEM performance and detection accuracy in larger and more complex environments.

8. References

- **Amazon Web Services – EC2 Documentation**

Amazon Elastic Compute Cloud (EC2) documentation was referred to for understanding the deployment, configuration, and management of virtual machines used in the project. This source helped in designing the cloud infrastructure, selecting instance types, and configuring security groups and networking.

URL: <https://docs.aws.amazon.com/ec2/>

- **Amazon Web Services – Virtual Private Cloud (VPC) Documentation**

AWS VPC documentation was used to design the private cloud network, including subnets, route tables, and security group rules. This reference supported the implementation of network isolation between the Active Directory server, attacker machine, and SIEM server.

URL: <https://docs.aws.amazon.com/vpc/>

- **AWS Systems Manager (SSM) Documentation**

AWS Systems Manager documentation was referred to for securely accessing EC2 instances using Session Manager and port forwarding, eliminating the need to expose RDP ports publicly.

URL: <https://docs.aws.amazon.com/systems-manager/>

- **Microsoft Active Directory Domain Services Documentation**

Microsoft's official documentation was used to understand the architecture, installation, and configuration of Active Directory Domain Services (AD DS). This reference was critical for setting up the Domain Controller, domain users, and authentication policies.

URL: <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/>

- **Microsoft Windows Security Event Logging Documentation**

This documentation was used to study Windows Security Event IDs such as **4624 (Successful Logon)** and **4625 (Failed Logon)**, which form the core detection logic for password spray attacks in the project.

URL: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625>

- **Splunk Enterprise Documentation**

Splunk documentation was used to configure log ingestion, indexing, search queries, and dashboards. It also helped in writing SPL queries for detecting suspicious authentication behavior.

URL: <https://docs.splunk.com/Documentation/Splunk>

- **Splunk Universal Forwarder Documentation**

This reference was used to install and configure the Splunk Universal Forwarder on Windows endpoints for forwarding Windows Event Logs and Sysmon data to the Splunk Enterprise server.

URL: <https://docs.splunk.com/Documentation/Forwarder>

- **Sysmon (System Monitor) Documentation – Microsoft Sysinternals**

Sysmon documentation was referred to for configuring detailed endpoint telemetry generation, including process creation and network connection logging, which enhanced detection capabilities.

URL: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>

- **MITRE ATT&CK Framework**

The MITRE ATT&CK framework was used to map the password spray attack technique to real-world adversary behavior. It provided theoretical grounding for attack simulation and detection logic.

URL: <https://attack.mitre.org/>

- **Kerbrute Tool Documentation**

Kerbrute documentation and GitHub repository were referenced to understand how Kerberos-based password spray attacks are executed against Active Directory environments.

URL: <https://github.com/ropnop/kerbrute>

- **NetExec (nxc) Documentation**

NetExec documentation was used to simulate SMB-based authentication attempts and validate failed credential attacks against Windows hosts.

URL: <https://github.com/Pennyw0rth/NetExec>

- **Atomic Red Team Documentation**

Atomic Red Team documentation was used to understand structured adversary simulation techniques and generate realistic attack telemetry for detection testing.

URL: <https://atomicredteam.io/>

- **Nmap Network Scanning Tool Documentation**

Nmap documentation was referenced to perform network reconnaissance and service enumeration during the attack simulation phase.

URL: <https://nmap.org/book/man.html>